

LAB ASSIGNMENT – SQL FUNCTIONS

APPLY SQL FUNCTIONS: STRING, NUMERIC, DATE, AND AGGREGATE FUNCTIONS

Name: Aditi M

USN: 1RUA25BCA0006

Course: BCA

1. AIM

To apply and demonstrate SQL String, Numeric, Date, and Aggregate Functions using a School Student Database.

2. OBJECTIVE

The objectives of this assignment are:

1. To understand the concept of SQL Functions.
2. To apply String Functions for manipulating text data.
3. To apply Numeric Functions for performing calculations.
4. To apply Date Functions for working with date values.
5. To apply Aggregate Functions for performing calculations on multiple rows.
6. To use the GROUP BY clause with aggregate functions.
7. To combine multiple SQL functions in a single query.

SQL functions accept input, perform an operation, and return an output.

3. DATABASE SCENARIO

A school database is used to maintain basic information about its students.

The database contains a table named **STUDENT** with information such as Student ID, Student Name, Class, Marks, Fee Paid, Date of Birth, and Admission Date.

4. CREATION OF STUDENT TABLE

INSERTING RECORDS INTO STUDENT TABLE

DISPLAYING THE STUDENT TABLE

The screenshot shows the OneCompiler MySQL Online Editor interface. The SQL editor on the left contains the following code:

```
1 CREATE TABLE STUDENT (  
2   Student_ID INT,  
3   Student_Name VARCHAR(50),  
4   Class VARCHAR(10),  
5   Marks INT,  
6   Fee_Paid DECIMAL(10,2),  
7   Date_of_Birth DATE,  
8   Admission_Date DATE  
9 );  
10 INSERT INTO STUDENT  
11 (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)  
12 VALUES  
13 (101, 'Kirthana', '10A', 85, 50000, '2008-05-15', '2023-06-10'),  
14 (102, 'Aditi', '10A', 92, 50000, '2008-11-20', '2023-06-12'),  
15 (103, 'Bhavya', '10B', 76, 45000, '2009-02-10', '2023-06-11'),  
16 (104, 'Harshitha', '10B', 88, 48000, '2008-08-25', '2023-06-15'),  
17 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');  
18 SELECT * FROM STUDENT;
```

The output window on the right displays the result of the SELECT query as a table:

Student_ID	Student_Name	Class	Marks	Fee_Paid	Date_of_Birth	Admission_Date
101	Kirthana	10A	85	50000.00	2008-05-15	2023-06-10
102	Aditi	10A	92	50000.00	2008-11-20	2023-06-12
103	Bhavya	10B	76	45000.00	2009-02-10	2023-06-11
104	Harshitha	10B	88	48000.00	2008-08-25	2023-06-15
105	Geetha	10A	65	40000.00	2009-01-30	2023-06-13

PART A – STRING FUNCTIONS

String functions are used to manipulate text data. In this table, Student_Name and Class are examples of text data

UPPER() FUNCTION

Purpose

The UPPER() function converts text into uppercase letters.

The screenshot shows the OneCompiler MySQL Online Editor interface. The SQL editor on the left contains the following code:

```
1 CREATE TABLE STUDENT (  
2   Student_ID INT,  
3   Student_Name VARCHAR(50),  
4   Class VARCHAR(10),  
5   Marks INT,  
6   Fee_Paid DECIMAL(10,2),  
7   Date_of_Birth DATE,  
8   Admission_Date DATE  
9 );  
10 INSERT INTO STUDENT  
11 (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)  
12 VALUES  
13 (101, 'Kirthana', '10A', 85, 50000, '2008-05-15', '2023-06-10'),  
14 (102, 'Aditi', '10A', 92, 50000, '2008-11-20', '2023-06-12'),  
15 (103, 'Bhavya', '10B', 76, 45000, '2009-02-10', '2023-06-11'),  
16 (104, 'Harshitha', '10B', 88, 48000, '2008-08-25', '2023-06-15'),  
17 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');  
18 SELECT * FROM STUDENT;  
19  
20  
21 SELECT Student_Name,  
22   UPPER(Student_Name) AS Uppercase_Name  
23 FROM STUDENT;
```

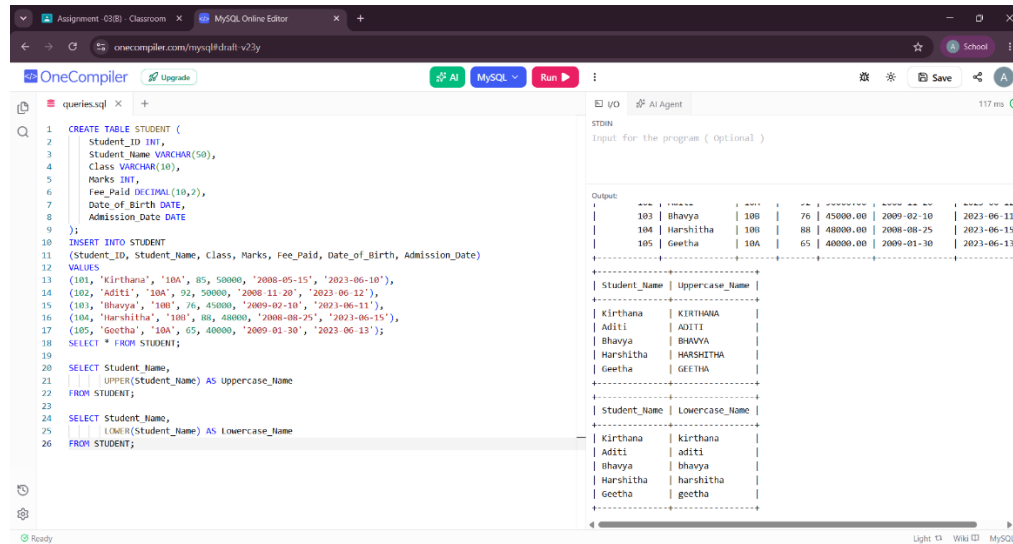
The output window on the right displays the result of the SELECT query as a table:

Student_Name	Uppercase_Name
Kirthana	KIRTHANA
Aditi	ADITI
Bhavya	BHAVYA
Harshitha	HARSHITHA
Geetha	GEETHA

LOWER() FUNCTION

Purpose

The LOWER() function converts text into lowercase letters.



The screenshot shows the OneCompiler MySQL Online Editor interface. The SQL editor on the left contains a query that creates a table named 'STUDENT' with columns: Student_ID INT, Student_Name VARCHAR(50), Class VARCHAR(10), Marks INT, Fee_Paid DECIMAL(10,2), Date_of_Birth DATE, and Admission_Date DATE. It then inserts five rows of student data and runs two queries: one to display all student records and another to display the same records with the Student_Name converted to uppercase using the UPPER() function.

```
1 CREATE TABLE STUDENT (  
2   Student_ID INT,  
3   Student_Name VARCHAR(50),  
4   Class VARCHAR(10),  
5   Marks INT,  
6   Fee_Paid DECIMAL(10,2),  
7   Date_of_Birth DATE,  
8   Admission_Date DATE  
9 );  
10 INSERT INTO STUDENT  
11 (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)  
12 VALUES  
13 (101, 'Kirthana', '10A', 85, 50000, '2008-05-15', '2023-06-10'),  
14 (102, 'Aditi', '10A', 92, 50000, '2008-11-20', '2023-06-12'),  
15 (103, 'Bhavya', '10A', 76, 45000, '2009-02-10', '2023-06-11'),  
16 (104, 'Harshitha', '10A', 88, 48000, '2008-08-25', '2023-06-15'),  
17 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');  
18 SELECT * FROM STUDENT;  
19  
20 SELECT Student_Name,  
21        UPPER(Student_Name) AS Uppercase_Name  
22 FROM STUDENT;  
23  
24 SELECT Student_Name,  
25        LOWER(Student_Name) AS Lowercase_Name  
26 FROM STUDENT;
```

The output on the right shows the results of the queries. The first query displays all student records. The second query displays the student names in uppercase.

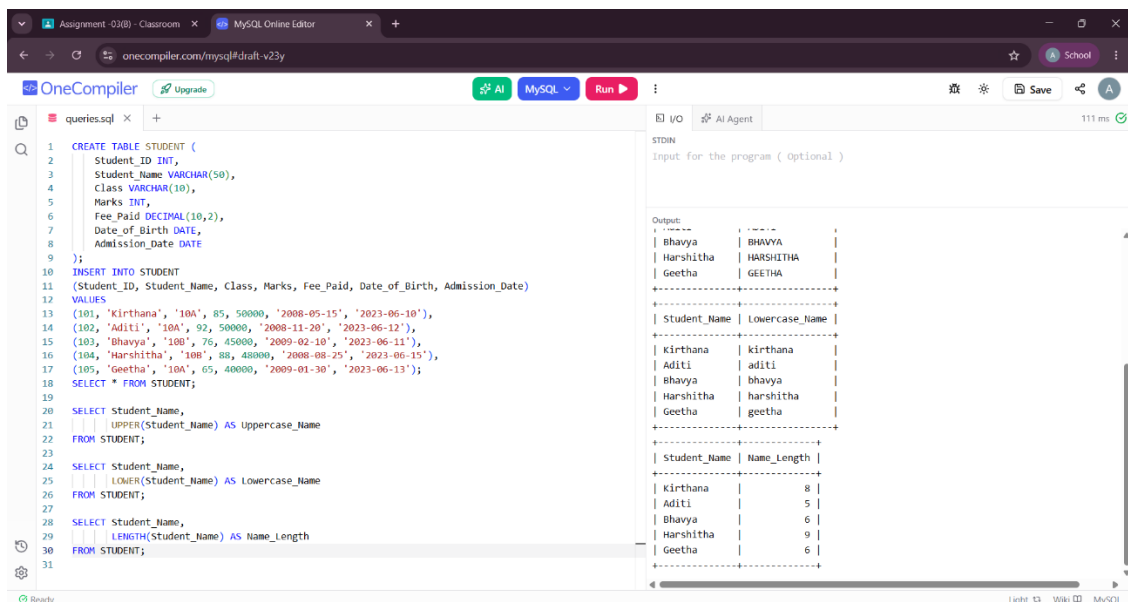
Student_ID	Student_Name	Class	Marks	Fee_Paid	Date_of_Birth	Admission_Date
101	Bhavya	10B	76	45000.00	2009-02-10	2023-06-11
104	Harshitha	10B	88	48000.00	2008-08-25	2023-06-15
105	Geetha	10A	65	40000.00	2009-01-30	2023-06-13

Student_Name	Uppercase_Name
Kirthana	KIRTHANA
Aditi	ADITI
Bhavya	BHAVYA
Harshitha	HARSHITHA
Geetha	GEETHA

LENGTH() FUNCTION

Purpose

The LENGTH() function finds the number of characters in a string.



The screenshot shows the OneCompiler MySQL Online Editor interface. The SQL editor on the left contains a query that creates a table named 'STUDENT' with columns: Student_ID INT, Student_Name VARCHAR(50), Class VARCHAR(10), Marks INT, Fee_Paid DECIMAL(10,2), Date_of_Birth DATE, and Admission_Date DATE. It then inserts five rows of student data and runs three queries: one to display all student records, one to display the student names in uppercase using the UPPER() function, and one to display the student names in lowercase using the LOWER() function. The final query uses the LENGTH() function to find the number of characters in each student name.

```
1 CREATE TABLE STUDENT (  
2   Student_ID INT,  
3   Student_Name VARCHAR(50),  
4   Class VARCHAR(10),  
5   Marks INT,  
6   Fee_Paid DECIMAL(10,2),  
7   Date_of_Birth DATE,  
8   Admission_Date DATE  
9 );  
10 INSERT INTO STUDENT  
11 (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)  
12 VALUES  
13 (101, 'Kirthana', '10A', 85, 50000, '2008-05-15', '2023-06-10'),  
14 (102, 'Aditi', '10A', 92, 50000, '2008-11-20', '2023-06-12'),  
15 (103, 'Bhavya', '10A', 76, 45000, '2009-02-10', '2023-06-11'),  
16 (104, 'Harshitha', '10A', 88, 48000, '2008-08-25', '2023-06-15'),  
17 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');  
18 SELECT * FROM STUDENT;  
19  
20 SELECT Student_Name,  
21        UPPER(Student_Name) AS Uppercase_Name  
22 FROM STUDENT;  
23  
24 SELECT Student_Name,  
25        LOWER(Student_Name) AS Lowercase_Name  
26 FROM STUDENT;  
27  
28 SELECT Student_Name,  
29        LENGTH(Student_Name) AS Name_Length  
30 FROM STUDENT;
```

The output on the right shows the results of the queries. The first query displays all student records. The second query displays the student names in uppercase. The third query displays the student names in lowercase. The final query displays the student names and their length in characters.

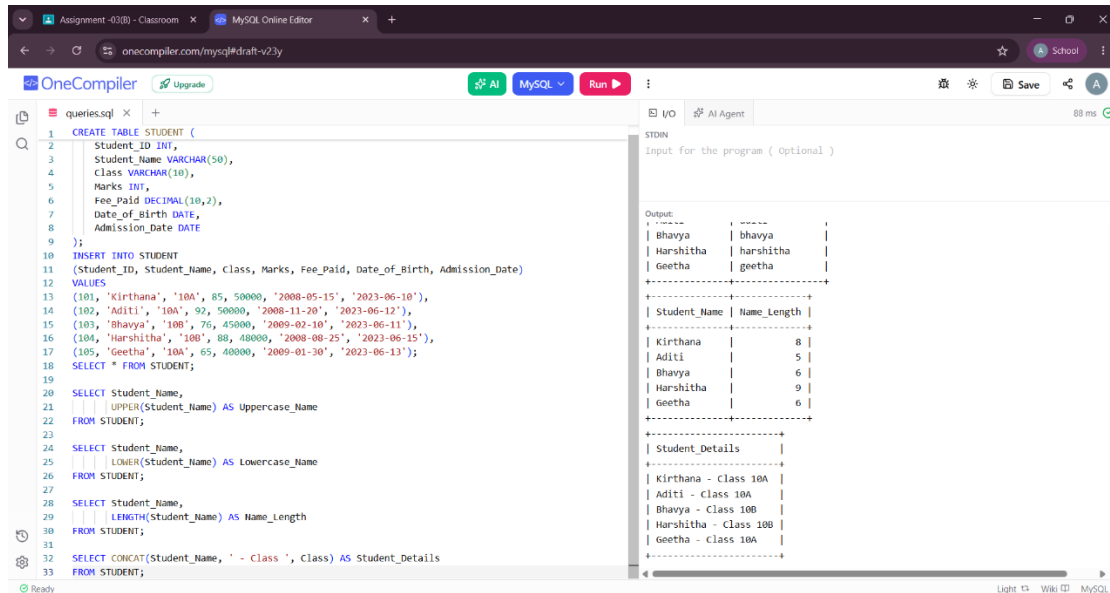
Student_Name	Lowercase_Name
Kirthana	kirthana
Aditi	aditi
Bhavya	bhavya
Harshitha	harshitha
Geetha	geetha

Student_Name	Name_Length
Kirthana	8
Aditi	5
Bhavya	6
Harshitha	9
Geetha	6

CONCAT() FUNCTION

Purpose

The **CONCAT()** function joins two or more strings together.



```
1 CREATE TABLE STUDENT (
2   Student_ID INT,
3   Student_Name VARCHAR(50),
4   Class VARCHAR(10),
5   Marks INT,
6   Fee_Paid DECIMAL(10,2),
7   Date_of_Birth DATE,
8   Admission_Date DATE
9 );
10 INSERT INTO STUDENT
11 (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)
12 VALUES
13 (101, 'Kirthana', '10A', 85, 50000, '2008-05-15', '2023-06-10'),
14 (102, 'Aditi', '10A', 92, 50000, '2008-11-20', '2023-06-12'),
15 (103, 'Bhavya', '10B', 76, 45000, '2009-02-10', '2023-06-11'),
16 (104, 'Harshitha', '10B', 88, 48000, '2008-08-25', '2023-06-15'),
17 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');
18 SELECT * FROM STUDENT;
19
20 SELECT Student_Name,
21        UPPER(Student_Name) AS Uppercase_Name
22 FROM STUDENT;
23
24 SELECT Student_Name,
25        LOWER(Student_Name) AS Lowercase_Name
26 FROM STUDENT;
27
28 SELECT Student_Name,
29        LENGTH(Student_Name) AS Name_Length
30 FROM STUDENT;
31
32 SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
33 FROM STUDENT;
```

Output:

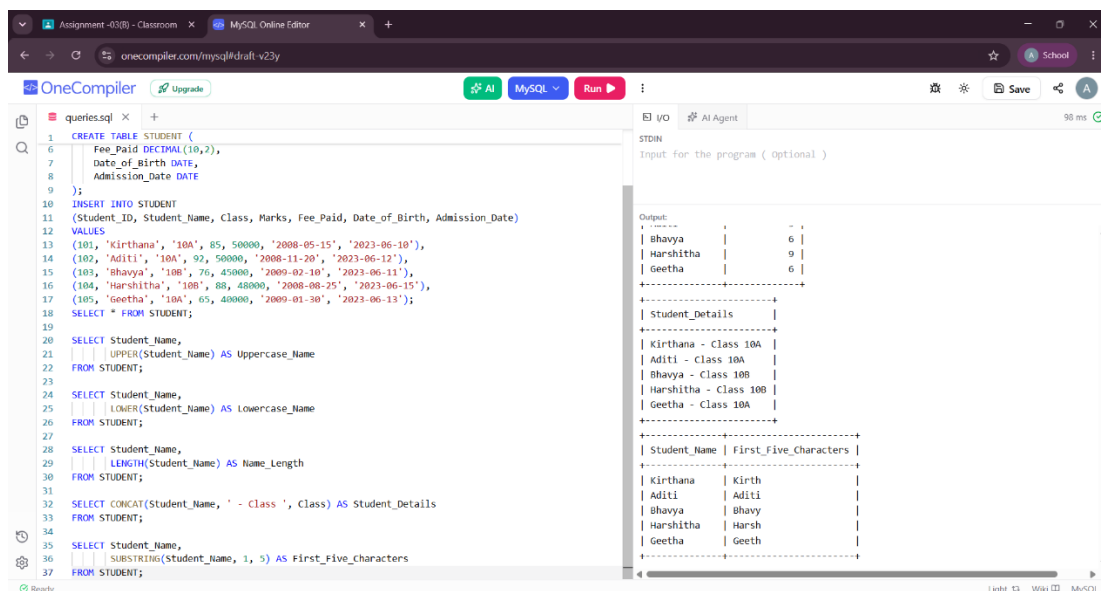
Student_Name	Name_Length
Kirthana	8
Aditi	5
Bhavya	6
Harshitha	9
Geetha	6

Student_Details
Kirthana - Class 10A
Aditi - Class 10A
Bhavya - Class 10B
Harshitha - Class 10B
Geetha - Class 10A

SUBSTRING() FUNCTION

Purpose

The **SUBSTRING()** function extracts a specific part of a string.



```
1 CREATE TABLE STUDENT (
2   Fee_Paid DECIMAL(10,2),
3   Date_of_Birth DATE,
4   Admission_Date DATE
5 );
6 INSERT INTO STUDENT
7 (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)
8 VALUES
9 (101, 'Kirthana', '10A', 85, 50000, '2008-05-15', '2023-06-10'),
10 (102, 'Aditi', '10A', 92, 50000, '2008-11-20', '2023-06-12'),
11 (103, 'Bhavya', '10B', 76, 45000, '2009-02-10', '2023-06-11'),
12 (104, 'Harshitha', '10B', 88, 48000, '2008-08-25', '2023-06-15'),
13 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');
14 SELECT * FROM STUDENT;
15
16 SELECT Student_Name,
17        UPPER(Student_Name) AS Uppercase_Name
18 FROM STUDENT;
19
20 SELECT Student_Name,
21        LOWER(Student_Name) AS Lowercase_Name
22 FROM STUDENT;
23
24 SELECT Student_Name,
25        LENGTH(Student_Name) AS Name_Length
26 FROM STUDENT;
27
28 SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
29 FROM STUDENT;
30
31 SELECT Student_Name,
32        SUBSTRING(Student_Name, 1, 5) AS First_Five_Characters
33 FROM STUDENT;
```

Output:

Student_Name	First_Five_Characters
Kirthana	Kirth
Aditi	Aditi
Bhavya	Bhavy
Harshitha	Harsh
Geetha	Geeth

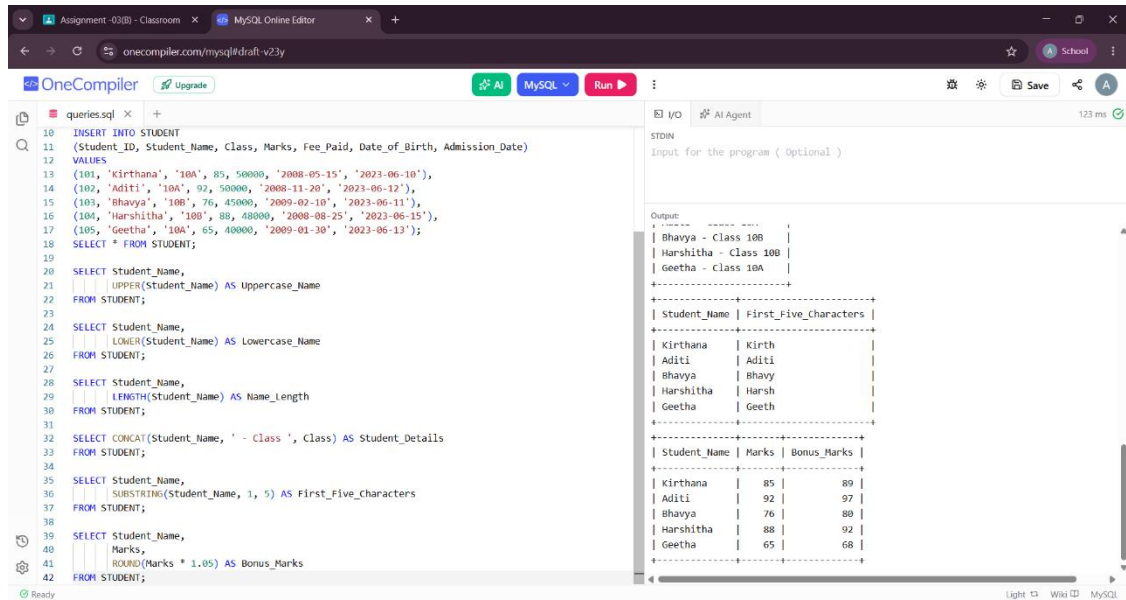
PART B – NUMERIC FUNCTIONS

Numeric functions perform calculations on numerical data such as **Marks** and **Fee_Paid**.

ROUND() FUNCTION

Purpose

The ROUND() function rounds a numerical value to the nearest integer.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a SQL script with the following queries:

```
10 INSERT INTO STUDENT
11 (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)
12 VALUES
13 (101, 'Kirthana', '10A', 85, 50000, '2008-05-15', '2023-06-10'),
14 (102, 'Aditi', '10A', 92, 50000, '2008-11-20', '2023-06-12'),
15 (103, 'Bhavya', '10B', 76, 45000, '2009-02-10', '2023-06-11'),
16 (104, 'Harshitha', '10B', 88, 48000, '2008-08-25', '2023-06-15'),
17 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');
18 SELECT * FROM STUDENT;
19
20 SELECT Student_Name,
21        UPPER(Student_Name) AS Uppercase_Name
22 FROM STUDENT;
23
24 SELECT Student_Name,
25        LOWER(Student_Name) AS Lowercase_Name
26 FROM STUDENT;
27
28 SELECT Student_Name,
29        LENGTH(Student_Name) AS Name_Length
30 FROM STUDENT;
31
32 SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
33 FROM STUDENT;
34
35 SELECT Student_Name,
36        SUBSTRING(Student_Name, 1, 5) AS First_Five_Characters
37 FROM STUDENT;
38
39 SELECT Student_Name,
40        Marks,
41        ROUND(Marks * 1.05) AS Bonus_Marks
42 FROM STUDENT;
```

The right pane shows the output of the queries:

Output:

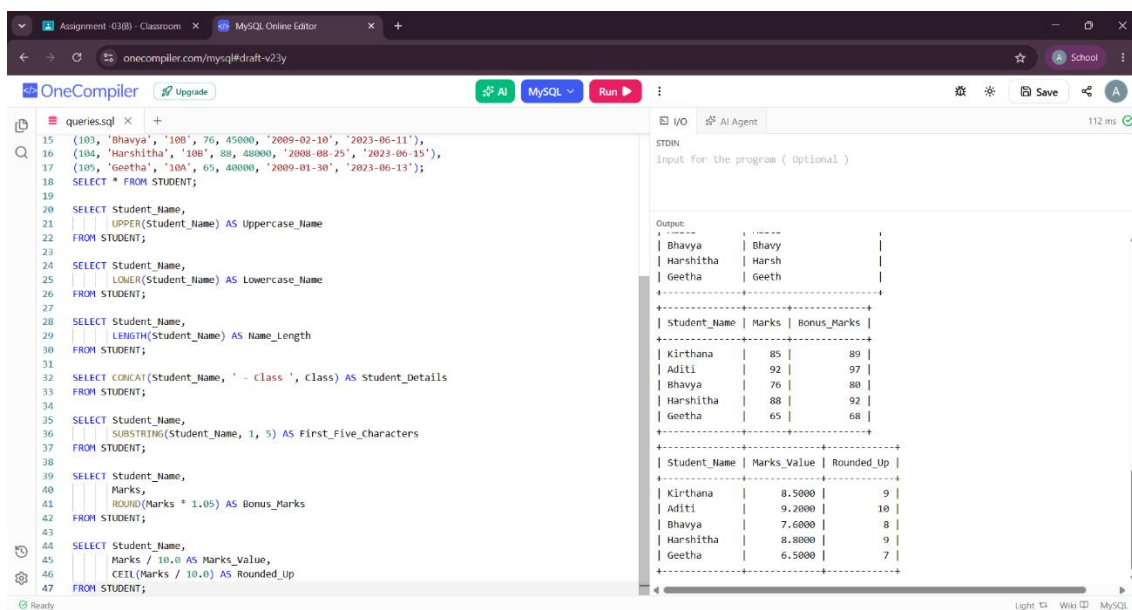
Student_Name	First_Five_Characters
Kirthana	Kirth
Aditi	Aditi
Bhavya	Bhavy
Harshitha	Harsh
Geetha	Geeth

Student_Name	Marks	Bonus_Marks
Kirthana	85	89
Aditi	92	97
Bhavya	76	80
Harshitha	88	92
Geetha	65	68

CEIL() FUNCTION

Purpose

The CEIL() function rounds a number upward to the next integer.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a SQL script with the following queries:

```
15 (103, 'Bhavya', '10B', 76, 45000, '2009-02-10', '2023-06-11'),
16 (104, 'Harshitha', '10B', 88, 48000, '2008-08-25', '2023-06-15'),
17 (105, 'Geetha', '10A', 65, 40000, '2009-01-30', '2023-06-13');
18 SELECT * FROM STUDENT;
19
20 SELECT Student_Name,
21        UPPER(Student_Name) AS Uppercase_Name
22 FROM STUDENT;
23
24 SELECT Student_Name,
25        LOWER(Student_Name) AS Lowercase_Name
26 FROM STUDENT;
27
28 SELECT Student_Name,
29        LENGTH(Student_Name) AS Name_Length
30 FROM STUDENT;
31
32 SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
33 FROM STUDENT;
34
35 SELECT Student_Name,
36        SUBSTRING(Student_Name, 1, 5) AS First_Five_Characters
37 FROM STUDENT;
38
39 SELECT Student_Name,
40        Marks,
41        ROUND(Marks * 1.05) AS Bonus_Marks
42 FROM STUDENT;
43
44 SELECT Student_Name,
45        Marks / 10.0 AS Marks_Value,
46        CEIL(Marks / 10.0) AS Rounded_Up
47 FROM STUDENT;
```

The right pane shows the output of the queries:

Output:

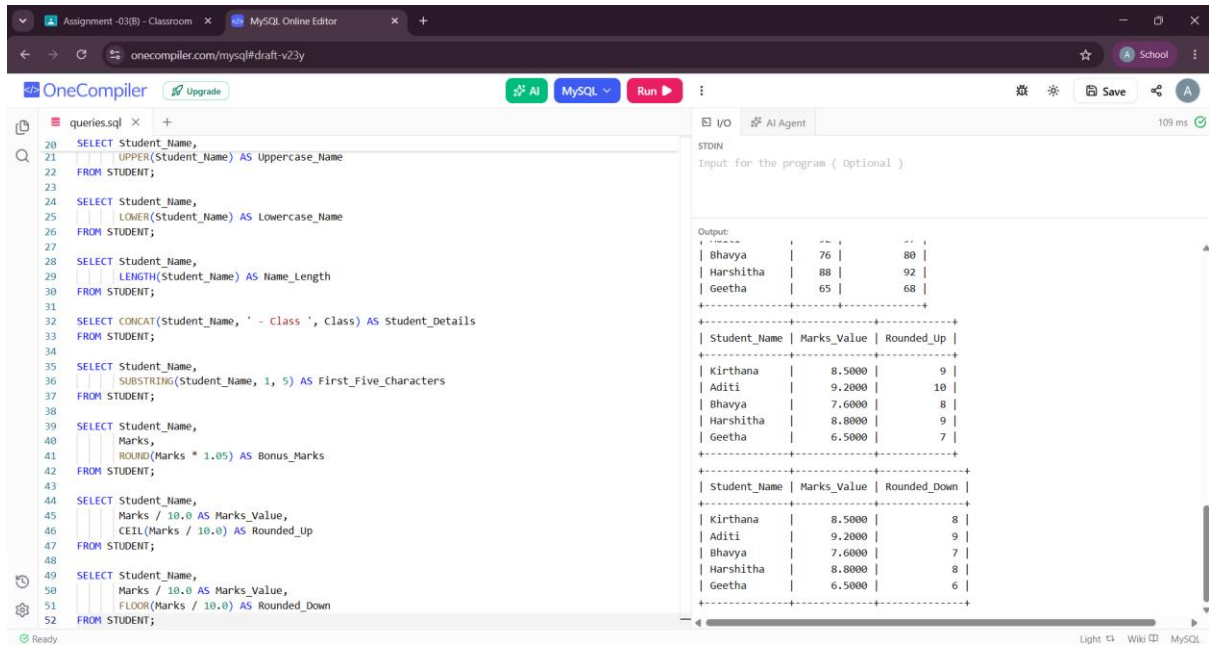
Student_Name	Marks	Bonus_Marks
Kirthana	85	89
Aditi	92	97
Bhavya	76	80
Harshitha	88	92
Geetha	65	68

Student_Name	Marks_Value	Rounded_Up
Kirthana	8.5000	9
Aditi	9.2000	10
Bhavya	7.6000	8
Harshitha	8.8000	9
Geetha	6.5000	7

FLOOR() FUNCTION

Purpose

The FLOOR() function rounds a number downward to the nearest integer.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a list of SQL queries, and the right pane shows the output of the selected query (query 52).

```
20 SELECT Student_Name,
21        UPPER(Student_Name) AS Uppercase_Name
22 FROM STUDENT;
23
24 SELECT Student_Name,
25        LOWER(Student_Name) AS Lowercase_Name
26 FROM STUDENT;
27
28 SELECT Student_Name,
29        LENGTH(Student_Name) AS Name_Length
30 FROM STUDENT;
31
32 SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
33 FROM STUDENT;
34
35 SELECT Student_Name,
36        SUBSTRING(Student_Name, 1, 5) AS First_Five_Characters
37 FROM STUDENT;
38
39 SELECT Student_Name,
40        Marks,
41        ROUND(Marks * 1.05) AS Bonus_Marks
42 FROM STUDENT;
43
44 SELECT Student_Name,
45        Marks / 10.0 AS Marks_Value,
46        CEIL(Marks / 10.0) AS Rounded_Up
47 FROM STUDENT;
48
49 SELECT Student_Name,
50        Marks / 10.0 AS Marks_Value,
51        FLOOR(Marks / 10.0) AS Rounded_Down
52 FROM STUDENT;
```

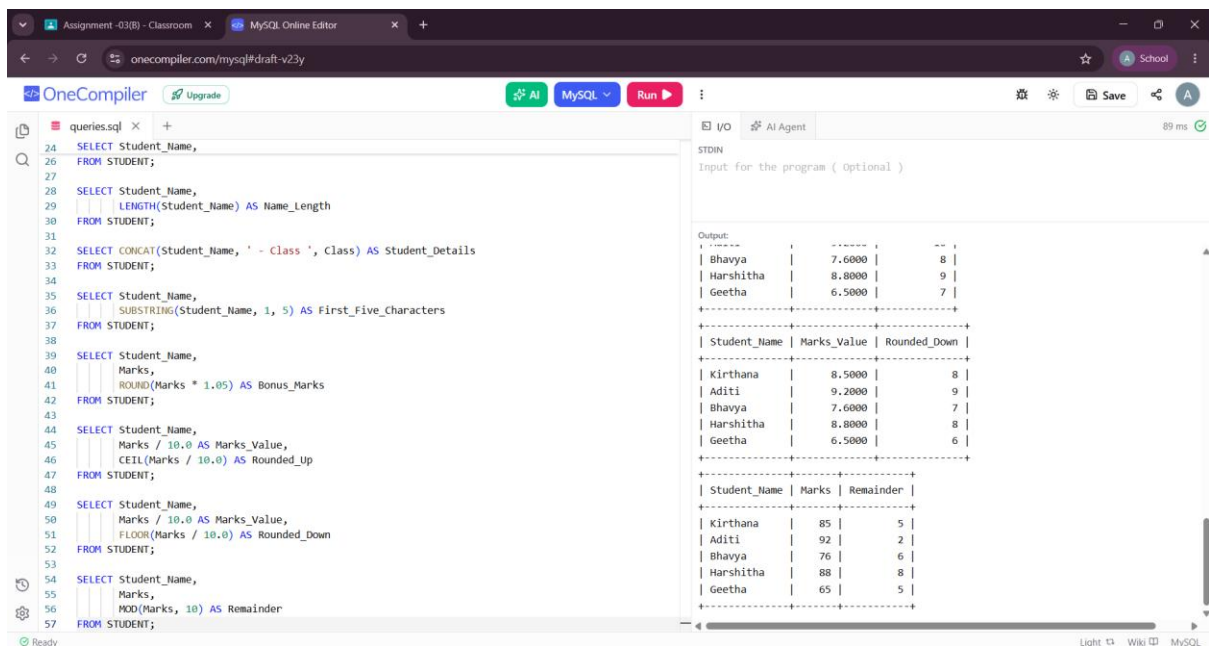
Output:

Student_Name	Marks_Value	Rounded_Down
Kirthana	8.5000	8
Aditi	9.2000	9
Bhavya	7.6000	7
Harshitha	8.8000	8
Geetha	6.5000	6

MOD() FUNCTION

Purpose

The MOD() function returns the remainder after division.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a list of SQL queries, and the right pane shows the output of the selected query (query 57).

```
24 SELECT Student_Name,
26 FROM STUDENT;
27
28 SELECT Student_Name,
29        LENGTH(Student_Name) AS Name_Length
30 FROM STUDENT;
31
32 SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
33 FROM STUDENT;
34
35 SELECT Student_Name,
36        SUBSTRING(Student_Name, 1, 5) AS First_Five_Characters
37 FROM STUDENT;
38
39 SELECT Student_Name,
40        Marks,
41        ROUND(Marks * 1.05) AS Bonus_Marks
42 FROM STUDENT;
43
44 SELECT Student_Name,
45        Marks / 10.0 AS Marks_Value,
46        CEIL(Marks / 10.0) AS Rounded_Up
47 FROM STUDENT;
48
49 SELECT Student_Name,
50        Marks / 10.0 AS Marks_Value,
51        FLOOR(Marks / 10.0) AS Rounded_Down
52 FROM STUDENT;
53
54 SELECT Student_Name,
55        Marks,
56        MOD(Marks, 10) AS Remainder
57 FROM STUDENT;
```

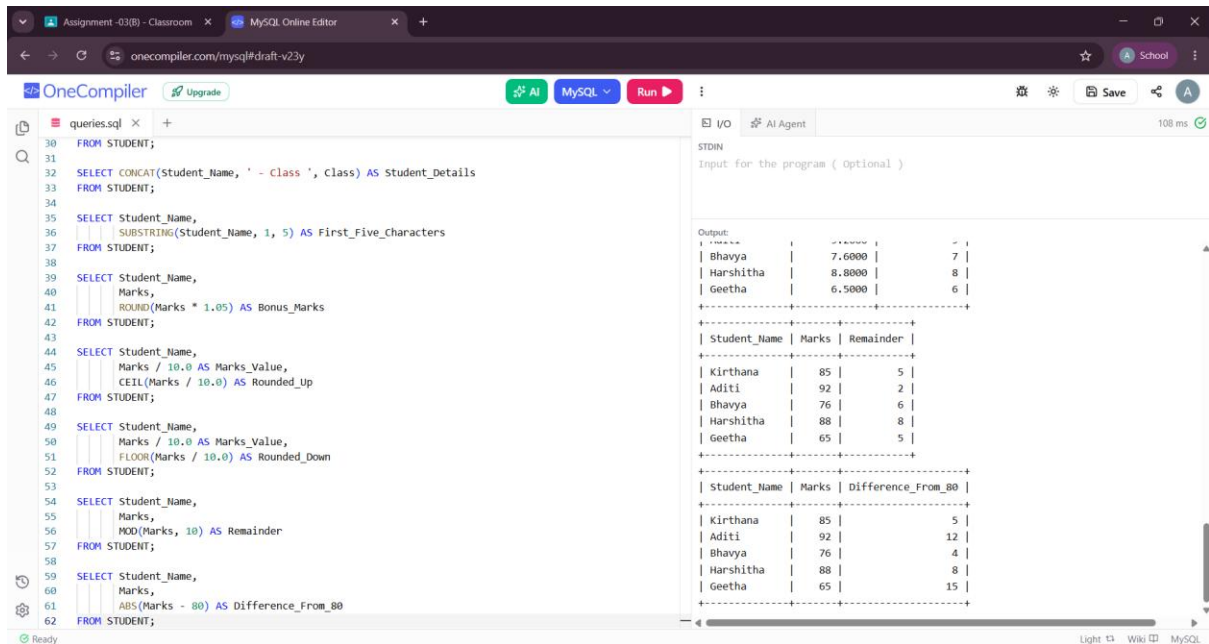
Output:

Student_Name	Marks	Remainder
Kirthana	85	5
Aditi	92	2
Bhavya	76	6
Harshitha	88	8
Geetha	65	5

ABS() FUNCTION

Purpose

The ABS() function returns the absolute value of a number. It can be used to find the difference between marks and a target value.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a series of SQL queries, and the right pane shows the output of the last query.

```
30 FROM STUDENT;
31
32 SELECT CONCAT(Student_Name, ' - Class ', Class) AS Student_Details
33 FROM STUDENT;
34
35 SELECT Student_Name,
36        SUBSTRING(Student_Name, 1, 5) AS First_Five_Characters
37 FROM STUDENT;
38
39 SELECT Student_Name,
40        Marks,
41        ROUND(Marks * 1.05) AS Bonus_Marks
42 FROM STUDENT;
43
44 SELECT Student_Name,
45        Marks / 10.0 AS Marks_Value,
46        CEIL(Marks / 10.0) AS Rounded_Up
47 FROM STUDENT;
48
49 SELECT Student_Name,
50        Marks / 10.0 AS Marks_Value,
51        FLOOR(Marks / 10.0) AS Rounded_Down
52 FROM STUDENT;
53
54 SELECT Student_Name,
55        Marks,
56        MOD(Marks, 10) AS Remainder
57 FROM STUDENT;
58
59 SELECT Student_Name,
60        Marks,
61        ABS(Marks - 80) AS Difference_From_80
62 FROM STUDENT;
```

The output of the last query is as follows:

Student_Name	Marks	Difference_From_80
Kirithana	85	5
Aditi	92	12
Bhavya	76	4
Harshitha	88	8
Geetha	65	15

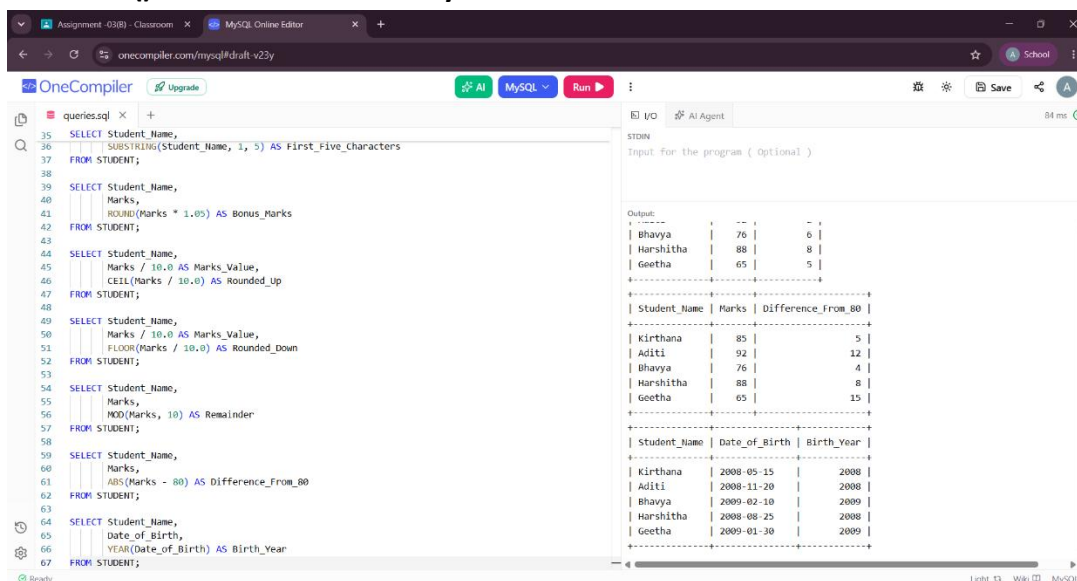
PART C – DATE FUNCTIONS

Date functions are used to work with date values such as Date of Birth and Admission Date.

YEAR() FUNCTION

Purpose

The YEAR() function extracts the year from a date.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a series of SQL queries, and the right pane shows the output of the last query.

```
35 SELECT Student_Name,
36        SUBSTRING(Student_Name, 1, 5) AS First_Five_Characters
37 FROM STUDENT;
38
39 SELECT Student_Name,
40        Marks,
41        ROUND(Marks * 1.05) AS Bonus_Marks
42 FROM STUDENT;
43
44 SELECT Student_Name,
45        Marks / 10.0 AS Marks_Value,
46        CEIL(Marks / 10.0) AS Rounded_Up
47 FROM STUDENT;
48
49 SELECT Student_Name,
50        Marks / 10.0 AS Marks_Value,
51        FLOOR(Marks / 10.0) AS Rounded_Down
52 FROM STUDENT;
53
54 SELECT Student_Name,
55        Marks,
56        MOD(Marks, 10) AS Remainder
57 FROM STUDENT;
58
59 SELECT Student_Name,
60        Marks,
61        ABS(Marks - 80) AS Difference_From_80
62 FROM STUDENT;
63
64 SELECT Student_Name,
65        Date_of_Birth,
66        YEAR(Date_of_Birth) AS Birth_Year
67 FROM STUDENT;
```

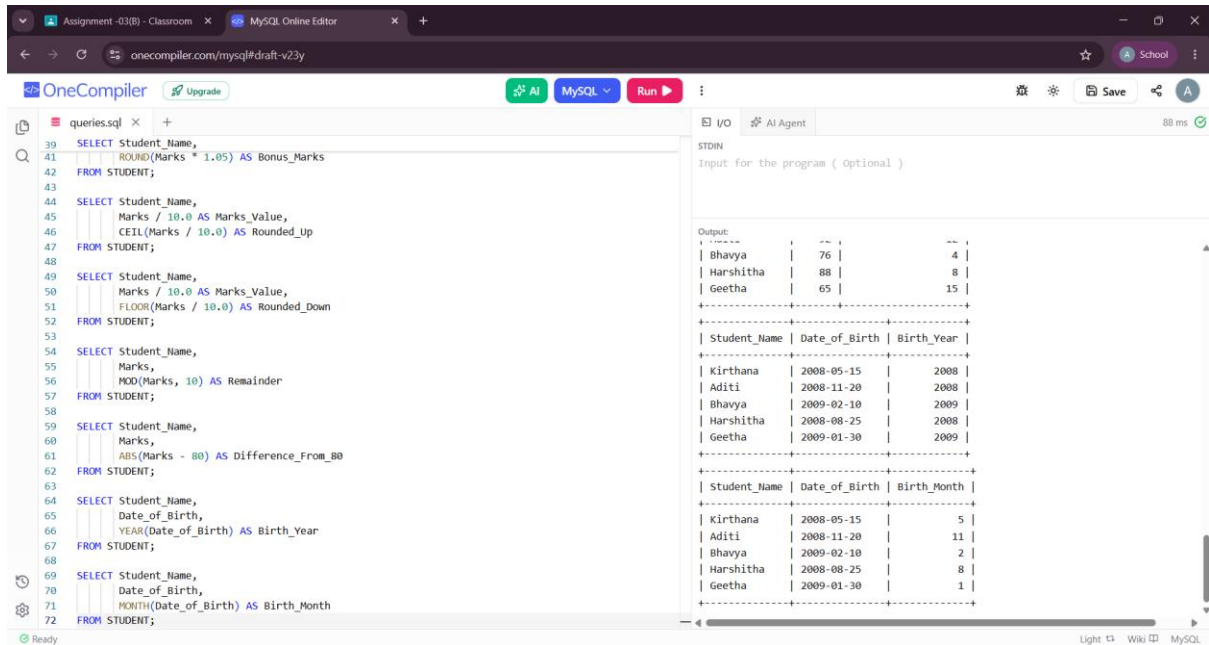
The output of the last query is as follows:

Student_Name	Date_of_Birth	Birth_Year
Kirithana	2008-05-15	2008
Aditi	2008-11-20	2008
Bhavya	2009-02-10	2009
Harshitha	2008-08-25	2008
Geetha	2009-01-30	2009

MONTH() FUNCTION

Purpose

The MONTH() function extracts the month from a date.



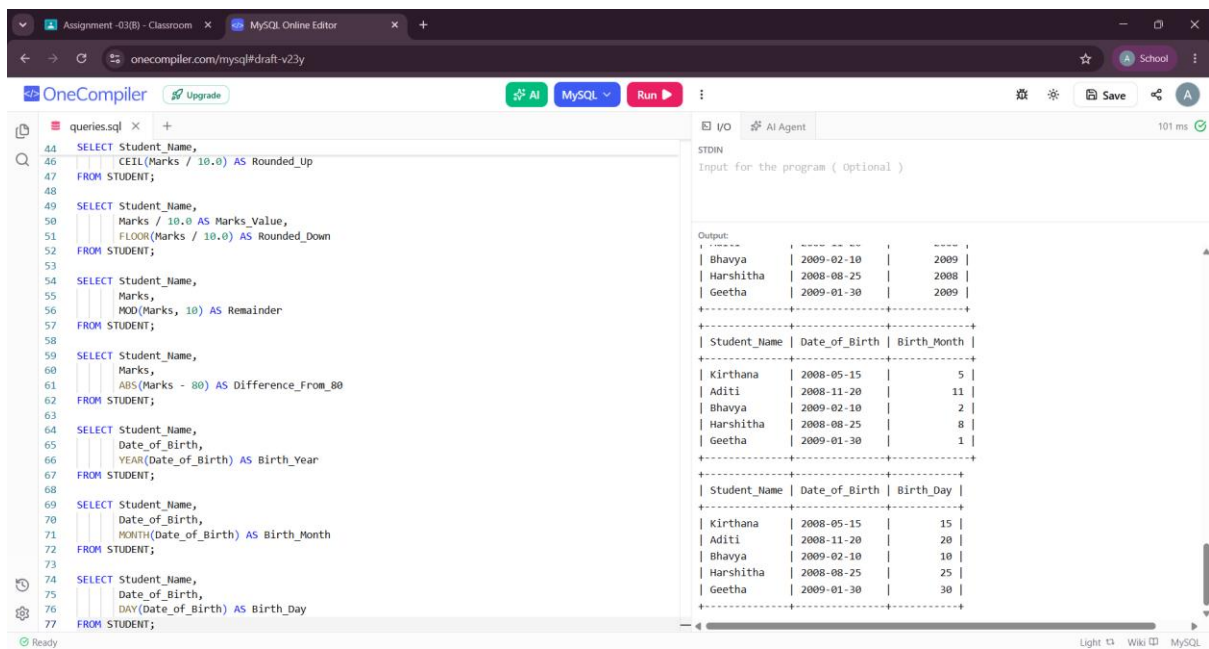
The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a list of SQL queries. The right pane shows the output of the selected query, which is the 72nd query: `SELECT Student_Name, Date_of_Birth, MONTH(Date_of_Birth) AS Birth_Month FROM STUDENT;`. The output is a table with 3 columns: Student_Name, Date_of_Birth, and Birth_Month. The data rows are as follows:

Student_Name	Date_of_Birth	Birth_Month
Kirthana	2008-05-15	5
Aditi	2008-11-20	11
Bhavya	2009-02-10	2
Harshitha	2008-08-25	8
Geetha	2009-01-30	1

DAY() FUNCTION

Purpose

The DAY() function extracts the day from a date.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a list of SQL queries. The right pane shows the output of the selected query, which is the 77th query: `SELECT Student_Name, Date_of_Birth, DAY(Date_of_Birth) AS Birth_Day FROM STUDENT;`. The output is a table with 3 columns: Student_Name, Date_of_Birth, and Birth_Day. The data rows are as follows:

Student_Name	Date_of_Birth	Birth_Day
Kirthana	2008-05-15	15
Aditi	2008-11-20	20
Bhavya	2009-02-10	10
Harshitha	2008-08-25	25
Geetha	2009-01-30	30

TIMESTAMPDIFF() FUNCTION

Purpose

The **TIMESTAMPDIFF()** function calculates the difference between two dates. It can be used to calculate the age of students.

The screenshot shows the OneCompiler MySQL Online Editor interface. The editor has a dark theme and a sidebar on the left with a file explorer showing 'queries.sql'. The main editor area contains a SQL script with line numbers 52 to 84. The script includes queries for selecting student names with marks, calculating remainders, differences, birth years, birth months, birth days, and the current date. The output pane on the right shows the results of the queries, including a table of student data and the current date.

SQL Script:

```

52 FROM STUDENT;
53
54 SELECT Student_Name,
55        Marks,
56        MOD(Marks, 10) AS Remainder
57 FROM STUDENT;
58
59 SELECT Student_Name,
60        Marks,
61        ABS(Marks - 80) AS Difference_From_80
62 FROM STUDENT;
63
64 SELECT Student_Name,
65        Date_of_Birth,
66        YEAR(Date_of_Birth) AS Birth_Year
67 FROM STUDENT;
68
69 SELECT Student_Name,
70        Date_of_Birth,
71        MONTH(Date_of_Birth) AS Birth_Month
72 FROM STUDENT;
73
74 SELECT Student_Name,
75        Date_of_Birth,
76        DAY(Date_of_Birth) AS Birth_Day
77 FROM STUDENT;
78
79 SELECT CURDATE() AS Today;
80
81 SELECT Student_Name,
82        Date_of_Birth,
83        TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age
84 FROM STUDENT;

```

Output:

Student_Name	Date_of_Birth	Birth_Day
Kirthana	2008-05-15	15
Aditi	2008-11-20	20
Bhavya	2009-02-10	10
Harshitha	2008-08-25	25
Geetha	2009-01-30	30

Today

2026-09-04

Student_Name | Date_of_Birth | Age

Kirthana	2008-05-15	18
Aditi	2008-11-20	17
Bhavya	2009-02-10	17
Harshitha	2008-08-25	18
Geetha	2009-01-30	17

CURDATE() FUNCTION

Purpose

The CURDATE() function returns the current date.

Assignment - 03(B) - Classroom MySQL Online Editor

queries.sql

```

47 FROM STUDENT;
48
49 SELECT Student_Name,
50        Marks / 10.0 AS Marks_Value,
51        FLOOR(Marks / 10.0) AS Rounded_Down
52 FROM STUDENT;
53
54 SELECT Student_Name,
55        Marks,
56        MOD(Marks, 10) AS Remainder
57 FROM STUDENT;
58
59 SELECT Student_Name,
60        Marks,
61        ABS(Marks - 80) AS Difference_From_80
62 FROM STUDENT;
63
64 SELECT Student_Name,
65        Date_of_Birth,
66        YEAR(Date_of_Birth) AS Birth_Year
67 FROM STUDENT;
68
69 SELECT Student_Name,
70        Date_of_Birth,
71        MONTH(Date_of_Birth) AS Birth_Month
72 FROM STUDENT;
73
74 SELECT Student_Name,
75        Date_of_Birth,
76        DAY(Date_of_Birth) AS Birth_Day
77 FROM STUDENT;
78
79 SELECT CURDATE() AS Today;

```

Output:

Student_Name	Date_of_Birth	Birth_Month
Kirthana	2008-05-15	5
Aditi	2008-11-20	11
Bhavya	2009-02-10	2
Harshitha	2008-08-25	8
Geetha	2009-01-30	1

Student_Name	Date_of_Birth	Birth_Day
Kirthana	2008-05-15	15
Aditi	2008-11-20	20
Bhavya	2009-02-10	10
Harshitha	2008-08-25	25
Geetha	2009-01-30	30

Student_Name	Date_of_Birth	Birth_Month
Today		

2026-09-04

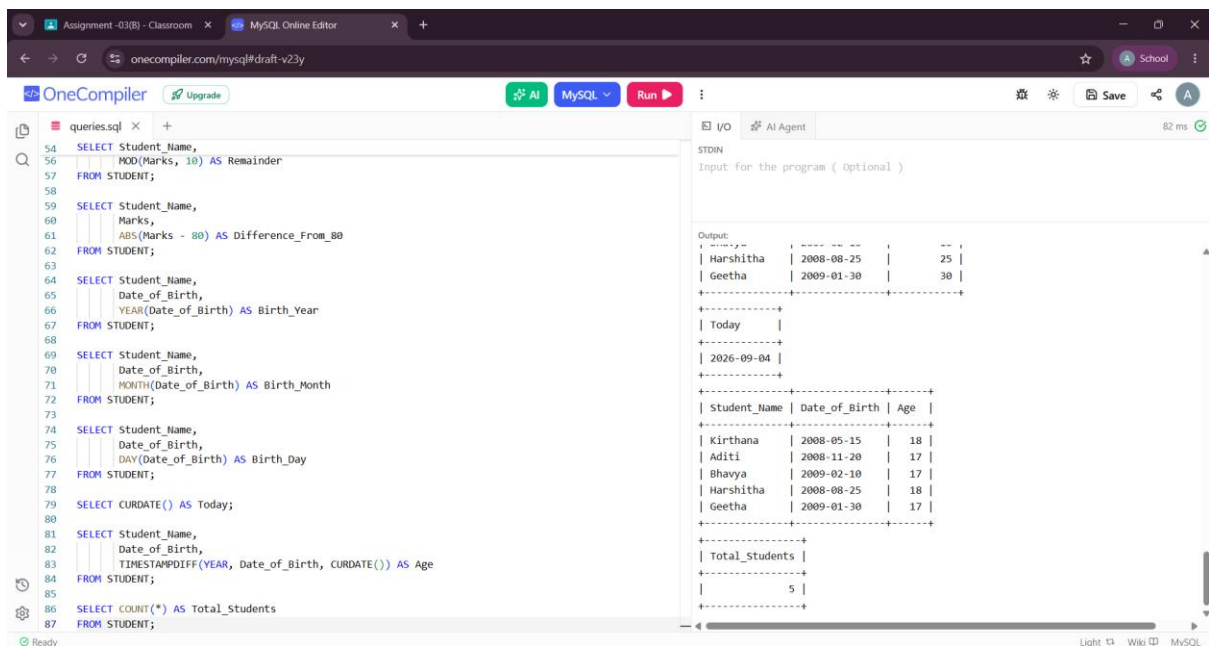
PART D – AGGREGATE FUNCTIONS

Aggregate functions operate on multiple rows together and produce a single result.

COUNT() FUNCTION

Purpose

The COUNT() function counts the number of records.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane displays a list of SQL queries, and the right pane shows the output of the selected query. The queries include:

- Query 54: `SELECT Student_Name, MOD(Marks, 10) AS Remainder FROM STUDENT;`
- Query 59: `SELECT Student_Name, Marks, ABS(Marks - 80) AS Difference_From_80 FROM STUDENT;`
- Query 64: `SELECT Student_Name, Date_of_Birth, YEAR(Date_of_Birth) AS Birth_Year FROM STUDENT;`
- Query 70: `SELECT Student_Name, Date_of_Birth, MONTH(Date_of_Birth) AS Birth_Month FROM STUDENT;`
- Query 74: `SELECT Student_Name, Date_of_Birth, DAY(Date_of_Birth) AS Birth_Day FROM STUDENT;`
- Query 79: `SELECT CURDATE() AS Today;`
- Query 81: `SELECT Student_Name, Date_of_Birth, TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age FROM STUDENT;`
- Query 86: `SELECT COUNT(*) AS Total_Students FROM STUDENT;`

The output of the selected query (Query 86) is shown in the right pane:

Student_Name	Date_of_Birth	Age
Kirthana	2008-05-15	18
Aditi	2008-11-20	17
Bhavya	2009-02-10	17
Harshitha	2008-08-25	18
Geetha	2009-01-30	17

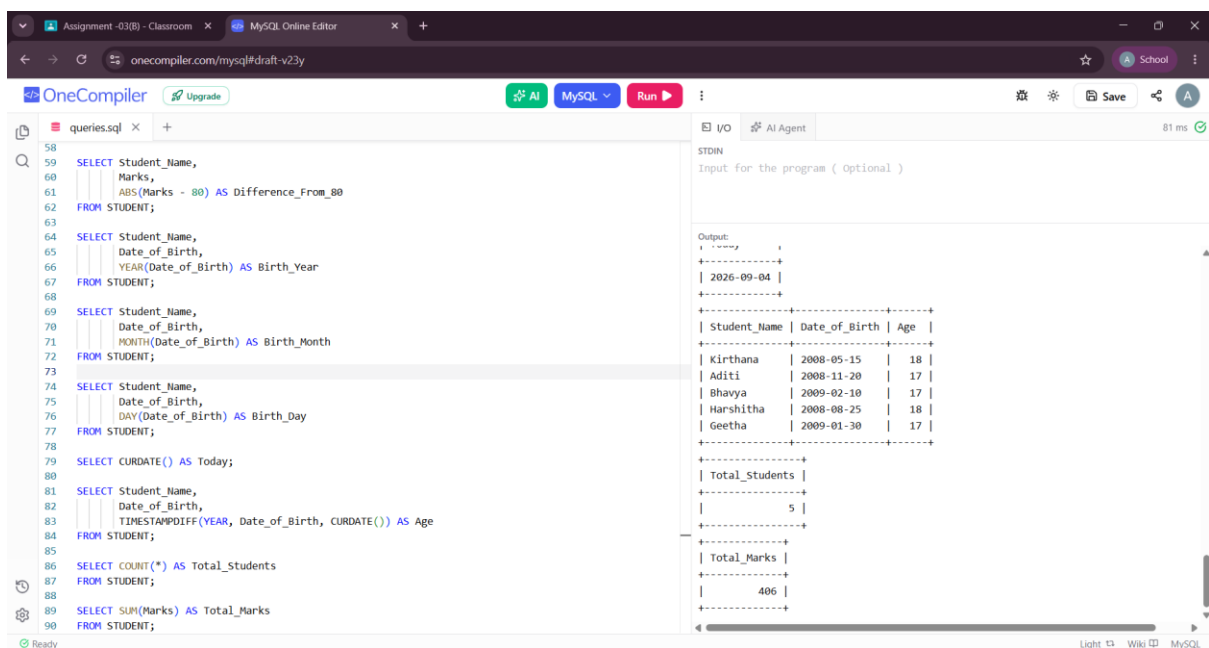
Below the table, the output shows the total number of students:

Total_Students
5

SUM() FUNCTION

Purpose

The SUM() function calculates the total of numerical values.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane displays a list of SQL queries, and the right pane shows the output of the selected query. The queries include:

- Query 59: `SELECT Student_Name, Marks, ABS(Marks - 80) AS Difference_From_80 FROM STUDENT;`
- Query 64: `SELECT Student_Name, Date_of_Birth, YEAR(Date_of_Birth) AS Birth_Year FROM STUDENT;`
- Query 70: `SELECT Student_Name, Date_of_Birth, MONTH(Date_of_Birth) AS Birth_Month FROM STUDENT;`
- Query 74: `SELECT Student_Name, Date_of_Birth, DAY(Date_of_Birth) AS Birth_Day FROM STUDENT;`
- Query 79: `SELECT CURDATE() AS Today;`
- Query 81: `SELECT Student_Name, Date_of_Birth, TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age FROM STUDENT;`
- Query 86: `SELECT COUNT(*) AS Total_Students FROM STUDENT;`
- Query 89: `SELECT SUM(Marks) AS Total_Marks FROM STUDENT;`

The output of the selected query (Query 89) is shown in the right pane:

Student_Name	Date_of_Birth	Age
Kirthana	2008-05-15	18
Aditi	2008-11-20	17
Bhavya	2009-02-10	17
Harshitha	2008-08-25	18
Geetha	2009-01-30	17

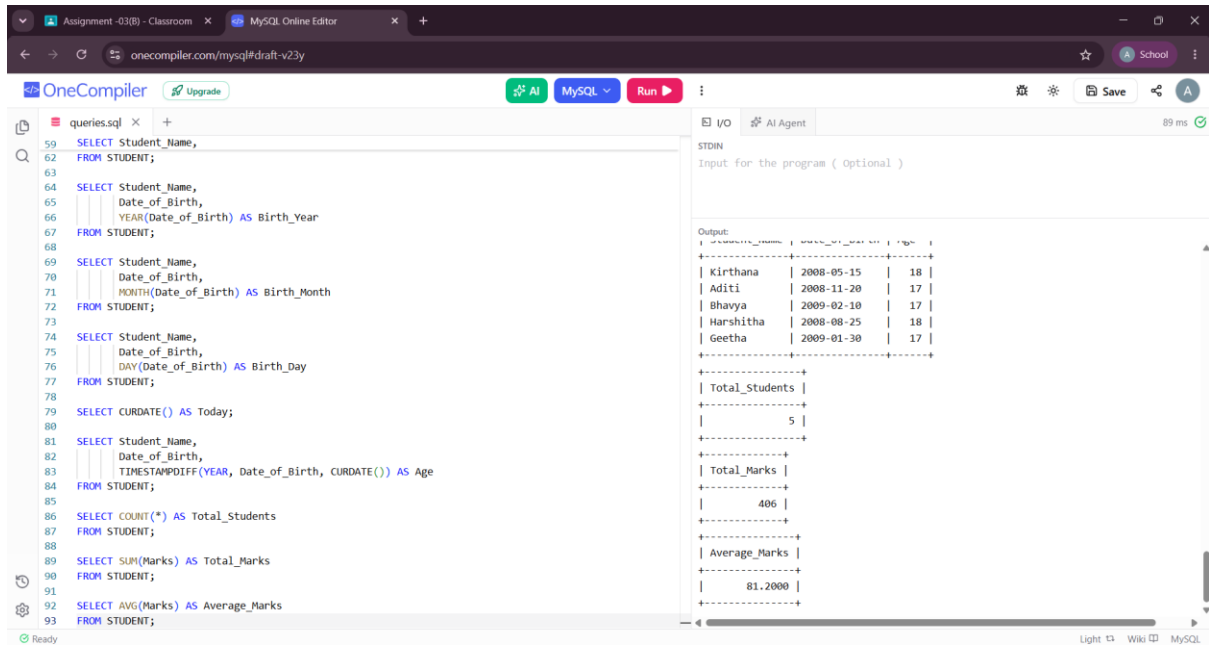
Below the table, the output shows the total marks:

Total_Marks
406

AVG() FUNCTION

Purpose

The AVG() function calculates the average of numerical values.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a list of SQL queries, and the right pane shows the output of the selected query (query 92).

```
59 SELECT Student_Name,  
62 FROM STUDENT;  
63  
64 SELECT Student_Name,  
65 Date_of_Birth,  
66 YEAR(Date_of_Birth) AS Birth_Year  
67 FROM STUDENT;  
68  
69 SELECT Student_Name,  
70 Date_of_Birth,  
71 MONTH(Date_of_Birth) AS Birth_Month  
72 FROM STUDENT;  
73  
74 SELECT Student_Name,  
75 Date_of_Birth,  
76 DAY(Date_of_Birth) AS Birth_Day  
77 FROM STUDENT;  
78  
79 SELECT CURDATE() AS Today;  
80  
81 SELECT Student_Name,  
82 Date_of_Birth,  
83 TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age  
84 FROM STUDENT;  
85  
86 SELECT COUNT(*) AS Total_Students  
87 FROM STUDENT;  
88  
89 SELECT SUM(Marks) AS Total_Marks  
90 FROM STUDENT;  
91  
92 SELECT AVG(Marks) AS Average_Marks  
93 FROM STUDENT;
```

Output:

Student_Name	Date_of_Birth	Age
Kirthana	2008-05-15	18
Aditi	2008-11-20	17
Bhavya	2009-02-10	17
Harshitha	2008-08-25	18
Geetha	2009-01-30	17

Total_Students
5

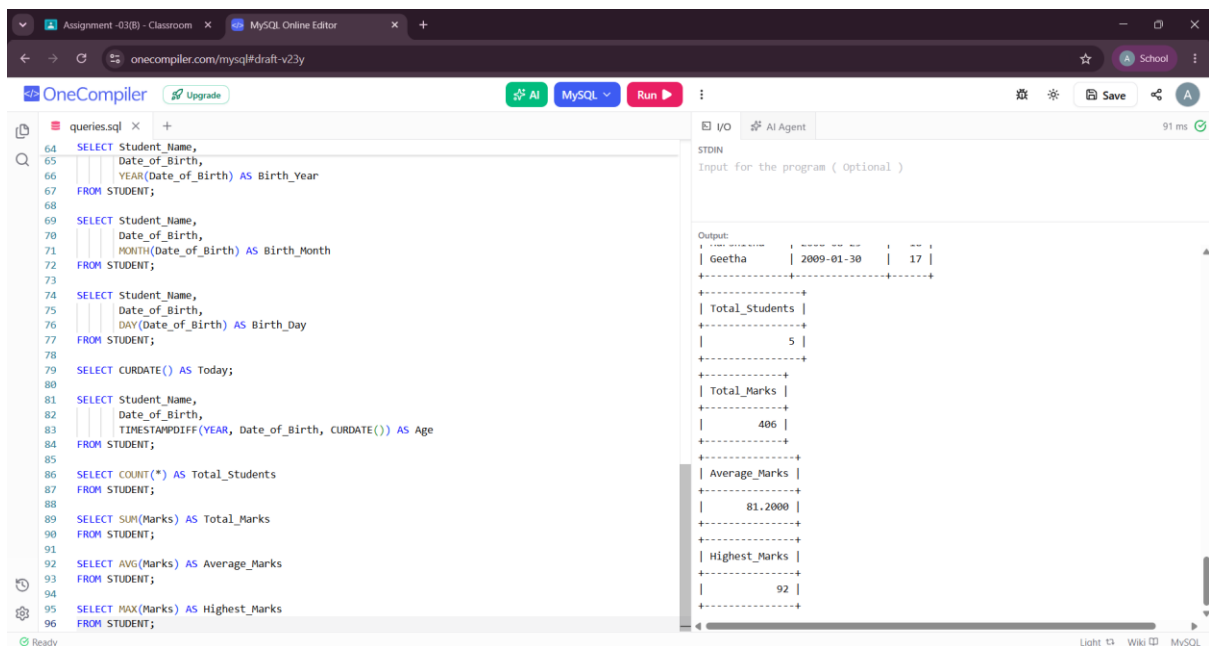
Total_Marks
406

Average_Marks
81.2000

MAX() FUNCTION

Purpose

The MAX() function finds the highest value from a column.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a list of SQL queries, and the right pane shows the output of the selected query (query 95).

```
64 SELECT Student_Name,  
65 Date_of_Birth,  
66 YEAR(Date_of_Birth) AS Birth_Year  
67 FROM STUDENT;  
68  
69 SELECT Student_Name,  
70 Date_of_Birth,  
71 MONTH(Date_of_Birth) AS Birth_Month  
72 FROM STUDENT;  
73  
74 SELECT Student_Name,  
75 Date_of_Birth,  
76 DAY(Date_of_Birth) AS Birth_Day  
77 FROM STUDENT;  
78  
79 SELECT CURDATE() AS Today;  
80  
81 SELECT Student_Name,  
82 Date_of_Birth,  
83 TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age  
84 FROM STUDENT;  
85  
86 SELECT COUNT(*) AS Total_Students  
87 FROM STUDENT;  
88  
89 SELECT SUM(Marks) AS Total_Marks  
90 FROM STUDENT;  
91  
92 SELECT AVG(Marks) AS Average_Marks  
93 FROM STUDENT;  
94  
95 SELECT MAX(Marks) AS Highest_Marks  
96 FROM STUDENT;
```

Output:

Student_Name	Date_of_Birth	Age
Geetha	2009-01-30	17

Total_Students
5

Total_Marks
406

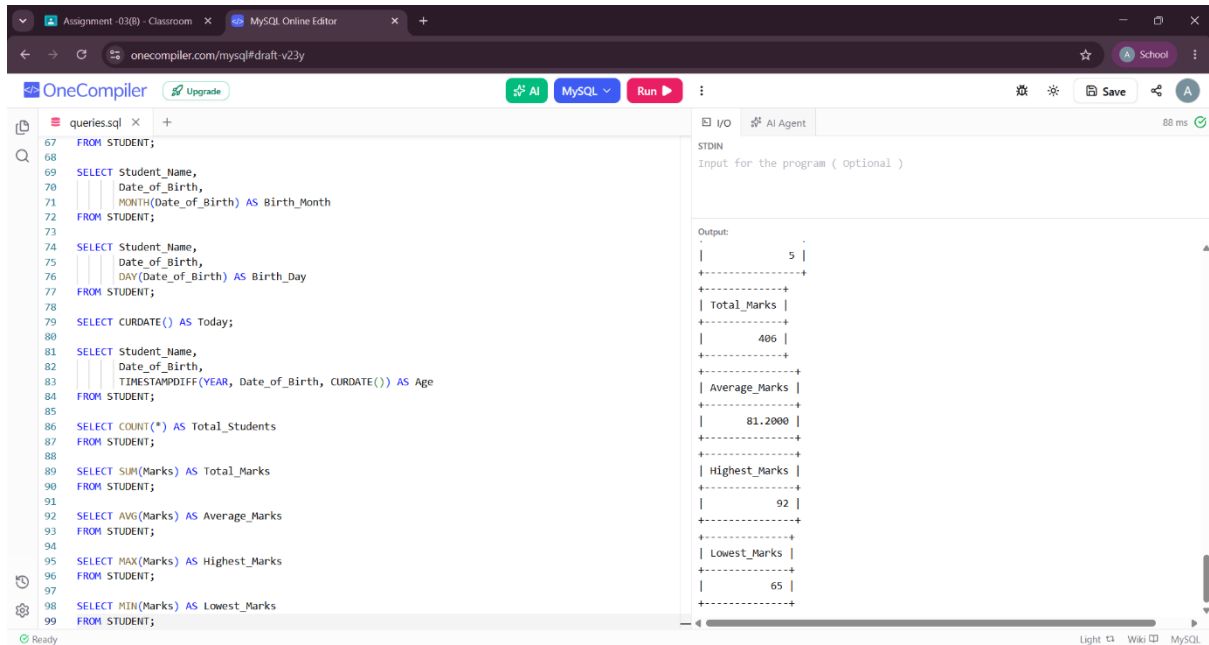
Average_Marks
81.2000

Highest_Marks
92

MIN() FUNCTION

Purpose

The MIN() function finds the lowest value from a column.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a SQL query with line numbers 67 to 99. The right pane shows the output of the query, which includes a table of student statistics.

```
67 FROM STUDENT;
68
69 SELECT Student_Name,
70        Date_of_Birth,
71        MONTH(Date_of_Birth) AS Birth_Month
72 FROM STUDENT;
73
74 SELECT Student_Name,
75        Date_of_Birth,
76        DAY(Date_of_Birth) AS Birth_Day
77 FROM STUDENT;
78
79 SELECT CURDATE() AS Today;
80
81 SELECT Student_Name,
82        Date_of_Birth,
83        TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age
84 FROM STUDENT;
85
86 SELECT COUNT(*) AS Total_Students
87 FROM STUDENT;
88
89 SELECT SUM(Marks) AS Total_Marks
90 FROM STUDENT;
91
92 SELECT AVG(Marks) AS Average_Marks
93 FROM STUDENT;
94
95 SELECT MAX(Marks) AS Highest_Marks
96 FROM STUDENT;
97
98 SELECT MIN(Marks) AS Lowest_Marks
99 FROM STUDENT;
```

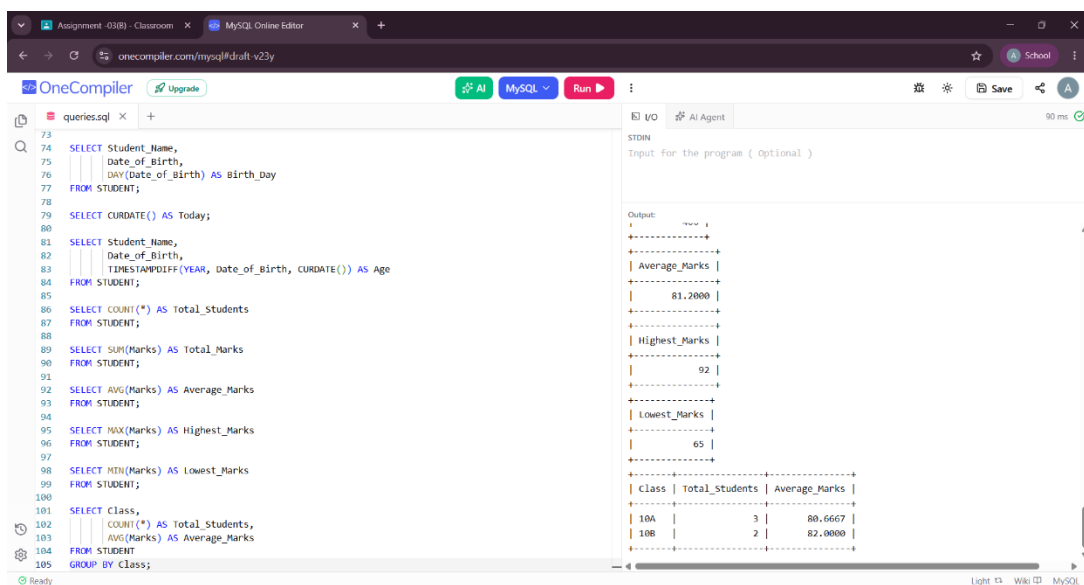
Output:

5
Total_Marks
406
Average_Marks
81.2000
Highest_Marks
92
Lowest_Marks
65

AGGREGATE FUNCTIONS USING GROUP BY

Purpose

The GROUP BY clause is used to group records based on a column. Here, students are grouped according to their class, and the number of students and average marks are calculated for each class.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane contains a SQL query with line numbers 73 to 105. The right pane shows the output of the query, which includes a table of student statistics grouped by class.

```
73
74 SELECT Student_Name,
75        Date_of_Birth,
76        DAY(Date_of_Birth) AS Birth_Day
77 FROM STUDENT;
78
79 SELECT CURDATE() AS Today;
80
81 SELECT Student_Name,
82        Date_of_Birth,
83        TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age
84 FROM STUDENT;
85
86 SELECT COUNT(*) AS Total_Students
87 FROM STUDENT;
88
89 SELECT SUM(Marks) AS Total_Marks
90 FROM STUDENT;
91
92 SELECT AVG(Marks) AS Average_Marks
93 FROM STUDENT;
94
95 SELECT MAX(Marks) AS Highest_Marks
96 FROM STUDENT;
97
98 SELECT MIN(Marks) AS Lowest_Marks
99 FROM STUDENT;
100
101 SELECT Class,
102        COUNT(*) AS Total_Students,
103        AVG(Marks) AS Average_Marks
104 FROM STUDENT
105 GROUP BY Class;
```

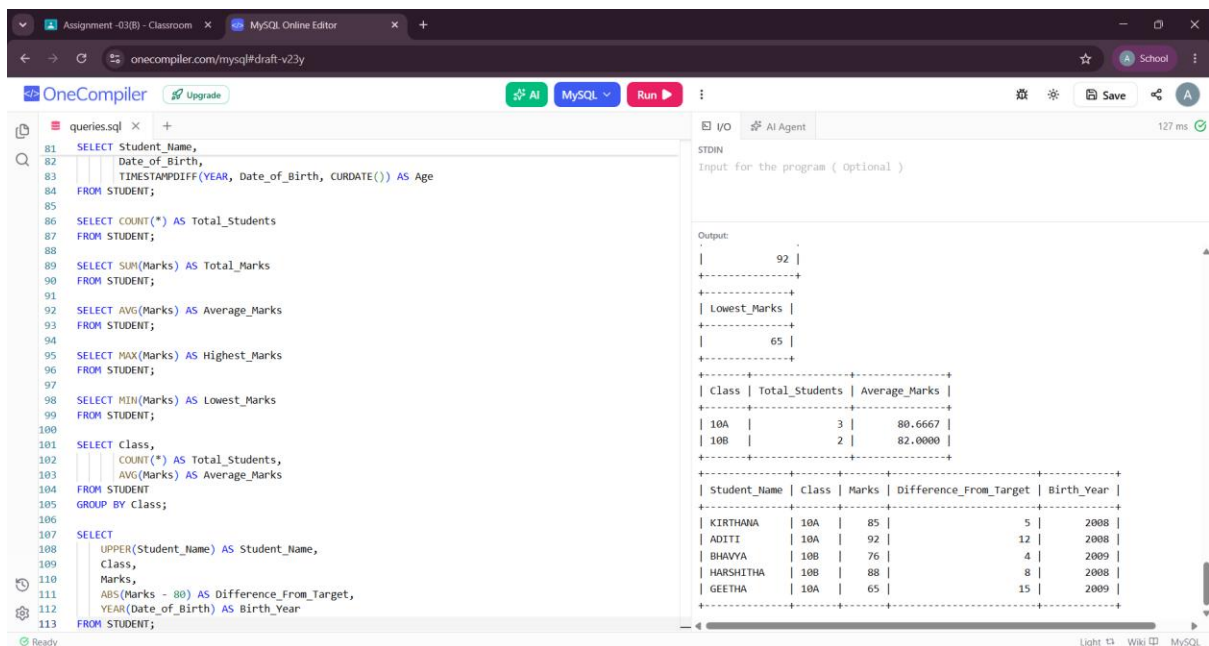
Output:

Average_Marks
81.2000
Highest_Marks
92
Lowest_Marks
65
Class Total_Students Average_Marks
10A 3 80.6667
10B 2 82.0000

COMBINED SQL FUNCTIONS

Purpose

This query combines multiple SQL functions to generate a student report containing the student's name in uppercase, class, marks, difference from the target mark of 80, and birth year.



The screenshot shows the OneCompiler MySQL Online Editor interface. The left pane displays a SQL query with line numbers 81 to 113. The right pane shows the output of the query, which includes several summary statistics and a detailed student report table.

```
81 SELECT Student_Name,
82        Date_of_Birth,
83        TIMESTAMPTDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age
84 FROM STUDENT;
85
86 SELECT COUNT(*) AS Total_Students
87 FROM STUDENT;
88
89 SELECT SUM(Marks) AS Total_Marks
90 FROM STUDENT;
91
92 SELECT AVG(Marks) AS Average_Marks
93 FROM STUDENT;
94
95 SELECT MAX(Marks) AS Highest_Marks
96 FROM STUDENT;
97
98 SELECT MIN(Marks) AS Lowest_Marks
99 FROM STUDENT;
100
101 SELECT Class,
102        COUNT(*) AS Total_Students,
103        AVG(Marks) AS Average_Marks
104 FROM STUDENT
105 GROUP BY Class;
106
107 SELECT
108        UPPER(Student_Name) AS Student_Name,
109        Class,
110        Marks,
111        ABS(Marks - 80) AS Difference_From_Target,
112        YEAR(Date_of_Birth) AS Birth_Year
113 FROM STUDENT;
```

Output:

```
STDIN
Input for the program ( optional )

Output:
+-----+
|          92          |
+-----+
+-----+
| Lowest_Marks |
+-----+
|          65          |
+-----+
+-----+
| Class | Total_Students | Average_Marks |
+-----+
| 10A   | 3              | 80.6667       |
| 10B   | 2              | 82.0000       |
+-----+
+-----+
| Student_Name | Class | Marks | Difference_From_Target | Birth_Year |
+-----+
| KIRTHANA     | 10A   | 85    | 5                      | 2008       |
| ADITI        | 10A   | 92    | 12                     | 2008       |
| BHAVYA       | 10B   | 76    | 4                      | 2009       |
| HARSHITHA    | 10B   | 88    | 8                      | 2008       |
| GEETHA       | 10A   | 65    | 15                     | 2009       |
+-----+
```

CONCLUSION

The SQL functions were successfully applied to the STUDENT database. String functions were used to manipulate text data, Numeric functions were used to perform calculations, Date functions were used to extract and calculate date-related information, and Aggregate functions were used to perform calculations on multiple records. The GROUP BY clause was also used to obtain class-wise results, and a combined query was executed using multiple SQL functions.